

ROUND ROBIN SCHEDULING

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, tq;
```

```
    int at[20], bt[20], rt[20];
```

```
    int ct[20], tat[20], wt[20], res[20];
```

```
    int time = 0, remain, flag = 0;
```

```
    // Arrays for Gantt chart
```

```
    int gcProcess[100], gcStart[100], gcEnd[100];
```

```
    int gcIndex = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(int i = 0; i < n; i++)
```

```
    {
```

```
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
        rt[i] = bt[i];
```

```
        res[i] = -1;
```

```
    }
```

```
    printf("Enter Time Quantum: ");
```

```
    scanf("%d", &tq);
```

```
    remain = n;
```

```

while(remain != 0)
{
    flag = 0;

    for(int i = 0; i < n; i++)
    {
        if(rt[i] > 0 && at[i] <= time)
        {
            if(res[i] == -1)
                res[i] = time - at[i];

            gcProcess[gcIndex] = i+1; // process ID
            gcStart[gcIndex] = time; // start time

            if(rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                ct[i] = time;
                rt[i] = 0;
                remain--;
            }

            gcEnd[gcIndex] = time; // end time
            gcIndex++;

            flag = 1;

```

```

    }
}

if(flag == 0)
    time++;
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], res[i]);
}

// Print Gantt Chart
printf("\nGantt Chart:\n");
for(int i = 0; i < gcIndex; i++)
{
    printf("| P%d ", gcProcess[i]);
}
printf("|\n");

for(int i = 0; i < gcIndex; i++)
{
    printf("%d\t", gcStart[i]);
}
printf("%d\n", gcEnd[gcIndex-1]);

```

```

return 0;
}

```

OUTPUT->

```

C:\Users\student\Desktop\RR X + v
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 1 2
Enter Arrival Time and Burst Time for P2: 2 3
Enter Arrival Time and Burst Time for P3: 3 1
Enter Arrival Time and Burst Time for P4: 5 3
Enter Time Quantum: 2

Process AT      BT      CT      TAT      WT      RT
P1      1      2      3      2      0      0
P2      2      3      9      7      4      1
P3      3      1      6      3      2      2
P4      5      3     10      5      2      1

Gantt Chart:
| P1 | P2 | P3 | P4 | P2 | P4 |
1      3      5      6      8      9      10

Process returned 0 (0x0)  execution time : 19.510 s
Press any key to continue.

```

MULTILEVEL QUEUE SCHEDULING

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, time = 0;
```

```
    int at[10], bt[10], ct[10], tat[10], wt[10];
```

```
    int type[10];
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```

{
    printf("Enter AT and BT for P%d: ", i+1);
    scanf("%d %d", &at[i], &bt[i]);

    printf("Enter type (1-System , 0-User): ");
    scanf("%d", &type[i]);
}

```

```

printf("\nExecution Order:\n");

```

```

for(i = 0; i < n; i++)
{
    if(type[i] == 1)
    {
        if(time < at[i])
            time = at[i];

        time += bt[i];
        ct[i] = time;

        printf("P%d ", i+1);
    }
}

```

```

for(i = 0; i < n; i++)
{
    if(type[i] == 0)

```

```

{
    if(time < at[i])
        time = at[i];

    time += bt[i];
    ct[i] = time;

    printf("P%d ", i+1);
}
}

printf("\n\nProcess\tAT\tBT\tType\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    if(type[i] == 1)
        printf("P%d\t%d\t%d\tSystem\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    else
        printf("P%d\t%d\t%d\tUser\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Output=

C:\Users\student\Desktop\mt X + v

```
Enter number of processes: 4
Enter AT and BT for P1: 1 2
Enter type (1-System , 0-User): 1
Enter AT and BT for P2: 2 4
Enter type (1-System , 0-User): 0
Enter AT and BT for P3: 3 5
Enter type (1-System , 0-User): 1
Enter AT and BT for P4: 4 2
Enter type (1-System , 0-User): 1
```

```
Execution Order:
P1 P3 P4 P2
```

Process	AT	BT	Type	CT	TAT	WT
P1	1	2	System	3	2	0
P2	2	4	User	14	12	8
P3	3	5	System	8	5	0
P4	4	2	System	10	6	4

```
Process returned 0 (0x0)   execution time : 54.618 s
Press any key to continue.
```